Exp12: Explore and implement Docker Swarm & compare with Kubernetes

What is Docker Swarm?

Docker Swarm is Docker's native container orchestration tool used to manage multiple containers across nodes.

Important Components

- ✓ Manager Node
- ✓ Worker Node
- ✓ Service
- ✓ Task (container instance)

Feature	**Docker Swarm**	**Kubernetes**
Setup	Very Easy	Complex
Learning Curve	Beginner-friendly	Steep
Scaling	Simple	Advanced
Networking	Simple	Powerful
Auto-healing	Basic	Strong
Industry Usage	Less	Very High
GUI	Limited	Many dashboards

Use **Docker Swarm** for:

- ✓ Small projects
- ✓ Learning
- ✓ Quick setup

Use **Kubernetes**:

- ✓ Production systems
- ✓ Large-scale applications
- ✓ Cloud environments

Step 1: Check Prerequisites

- ✓ Docker installed
- ✓ Docker service running

Check:

```
docker --version

mkdir exp12 && cd exp12
```

Step 2: Initialize Swarm

```
docker swarm init
```

Output:

```
Swarm initialized: current node is now a manager
```

Your system becomes a **Manager Node**

```
cse@cse-B760M-H:~/exp12$ docker swarm init
Swarm initialized: current node (1ze6gm9x1a9tteu0bfpwotgsy) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-3u0ua59w5ozk9fodwnutmw6zyjcgq1ftmoy8p535sqel9k894r-bref4y60xylsp6s0va4f0cwwv 172.17.4.121:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
cse@cse-B760M-H:~/exp12$
```

Step 2: Check Nodes

```
docker node ls
```

Expected output:

```
Leader (your machine)
```

```
cse@cse-B760M-H:~/exp12$ docker node ls
ID                            HOSTNAME      STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
1ze6gm9x1a9tteu0bfpwotgsy *   cse-B760M-H   Ready     Active         Leader           28.2.2
cse@cse-B760M-H:~/exp12$
```

Step 3: Create a Service

```
docker service create --name webapp -p 8080:80 nginx
```

This:

- ✓ Pulls nginx image
- ✓ Creates container(s)
- ✓ Exposes on port 8080

```
cse@cse-B760M-H:~/exp12$ docker service create --name webapp -p 8080:80 nginx
7sb48d0t5ciujn6nd85zptysa
overall progress: 1 out of 1 tasks
1/1: running   [==================================================>]
verify: Service 7sb48d0t5ciujn6nd85zptysa converged
cse@cse-B760M-H:~/exp12$
```

Step 4: Verify Service

```
docker service ls
```

```
docker service ps webapp
```

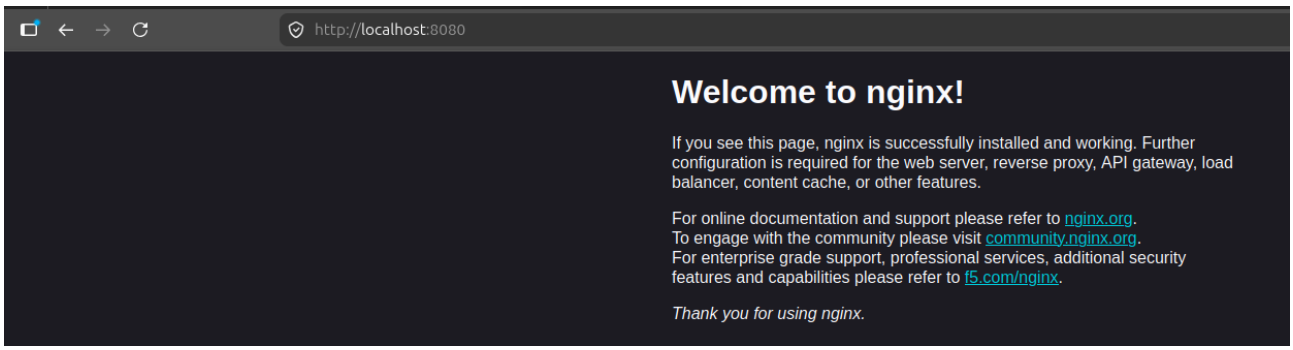
```
cse@cse-B760M-H:~/exp12$ docker service ls
ID             NAME      MODE         REPLICAS   IMAGE          PORTS
7sb48d0t5ciu   webapp    replicated   1/1        nginx:latest   *:8080->80/tcp
cse@cse-B760M-H:~/exp12$
cse@cse-B760M-H:~/exp12$
cse@cse-B760M-H:~/exp12$
cse@cse-B760M-H:~/exp12$ docker service ps webapp
ID             NAME       IMAGE          NODE          DESIRED STATE   CURRENT STATE            ERROR     PORTS
ow1l4banp4ic   webapp.1   nginx:latest   cse-B760M-H   Running         Running 50 seconds ago
cse@cse-B760M-H:~/exp12$
```

Step 5: Access Application

Open browser:

```
http://localhost:8080
```

Expected output:
Nginx welcome page



Step 6: Scale the Service

`docker service scale webapp=3`

```
cse@cse-B760M-H:~/exp12$ docker service scale webapp=3
webapp scaled to 3
overall progress: 3 out of 3 tasks
1/3: running   [==================================================>]
2/3: running   [==================================================>]
3/3: running   [==================================================>]
verify: Service webapp converged
cse@cse-B760M-H:~/exp12$
```

Check:

`docker service ps webapp`

Now 3 replicas running

```
cse@cse-B760M-H:~/exp12$ docker service ps webapp
ID             NAME       IMAGE          NODE          DESIRED STATE   CURRENT STATE            ERROR     PORTS
ow1l4banp4ic   webapp.1   nginx:latest   cse-B760M-H   Running         Running 2 minutes ago
lrdkq63kgxer   webapp.2   nginx:latest   cse-B760M-H   Running         Running 46 seconds ago
3bj4eaxaw42e   webapp.3   nginx:latest   cse-B760M-H   Running         Running 46 seconds ago
cse@cse-B760M-H:~/exp12$
```

Step 7: Observe Load Balancing

Refresh browser multiple times. Requests handled by different containers.

Right now all our replicas are serving the SAME default Nginx page.

So browser looks identical every time.

That is why we cannot visually observe load balancing.

Step 8: Prove Load Balancing Properly

We must make each container to display different content.

Then browser refresh will clearly show:

- ✓ container 1 response
- ✓ container 2 response
- ✓ container 3 response

Instead of default nginx image, let us create custom containers showing hostname.

Remove Existing Service

```
docker service rm webapp
```

```
cse@cse-B760M-H:~/exp12$ docker service rm webapp
webapp
cse@cse-B760M-H:~/exp12$
```

Create Service Showing Container Hostname

Run:

```
docker service create \
--name webapp \
-p 8080:80 \
--replicas 3 \
nginx sh -c "echo Container: \$(hostname) >
/usr/share/nginx/html/index.html && nginx -g 'daemon off;'"
```

```
cse@cse-B760M-H:~/exp12$ docker service create \
--name webapp \
-p 8080:80 \
--replicas 3 \
nginx sh -c "echo Container: \$(hostname) > /usr/share/nginx/html/index.html && nginx -g 'daemon off;'"
4t5mj5qqomyupp5cryee5e1p9
overall progress: 3 out of 3 tasks
1/3: running   [==================================================>]
2/3: running   [==================================================>]
3/3: running   [==================================================>]
verify: Service 4t5mj5qqomyupp5cryee5e1p9 converged
cse@cse-B760M-H:~/exp12$
```

Each container writes its own hostname into webpage.

Example:

```
Container: abc123
```

Another replica:

```
Container: xyz789
```

Open Browser

Open:

```
http://localhost:8080
```

Refresh multiple times.

Now we observe

Page changes like:

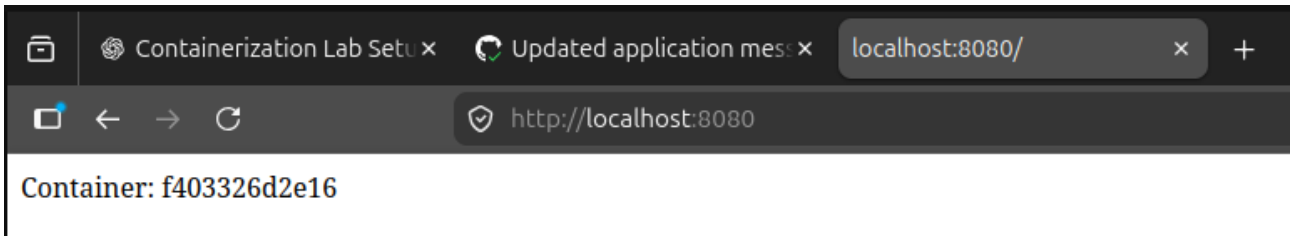
```
Container: x8k21
```

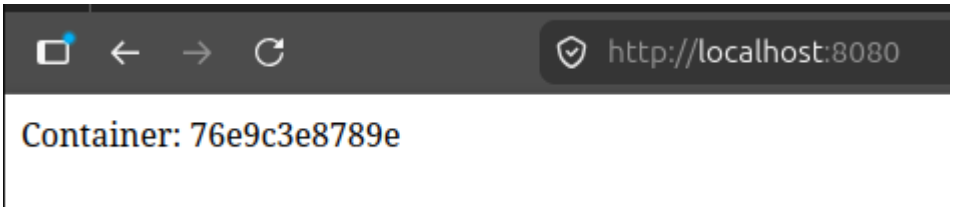
Refresh:

```
Container: p9l44
```

Refresh again:

```
Container: t7z91
```





Alternate method 1:

Run the following command in terminal and observe changes in itself.

```
curl localhost:8080
```

Alternate method 2:

```
docker service ls
```

```
docker service ps webapp
```

```
docker ps
```

```
cse@cse-B760M-H:~/exp12$ docker service ls
ID             NAME      MODE         REPLICAS   IMAGE          PORTS
4t5mj5qqomyu   webapp    replicated   3/3        nginx:latest   *:8080->80/tcp
cse@cse-B760M-H:~/exp12$
cse@cse-B760M-H:~/exp12$
cse@cse-B760M-H:~/exp12$ docker service ps webapp
ID             NAME       IMAGE          NODE          DESIRED STATE   CURRENT STATE           ERROR     PORTS
ttj29vp1pjop   webapp.1   nginx:latest   cse-B760M-H   Running         Running 4 minutes ago
k0hujql3mtt6   webapp.2   nginx:latest   cse-B760M-H   Running         Running 4 minutes ago
iinlgfap75vz   webapp.3   nginx:latest   cse-B760M-H   Running         Running 4 minutes ago
cse@cse-B760M-H:~/exp12$
cse@cse-B760M-H:~/exp12$
cse@cse-B760M-H:~/exp12$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS         PORTS     NAMES
f403326d2e16   nginx:latest   "/docker-entrypoint.…"   4 minutes ago   Up 4 minutes   80/tcp    webapp.2.k0hujql3mtt602h9sxrc58czp
743d12377763   nginx:latest   "/docker-entrypoint.…"   4 minutes ago   Up 4 minutes   80/tcp    webapp.1.ttj29vp1pjopv42d2163k5szi
76e9c3e8789e   nginx:latest   "/docker-entrypoint.…"   4 minutes ago   Up 4 minutes   80/tcp    webapp.3.iinlgfap75vzmphv00bxfe3fr
cse@cse-B760M-H:~/exp12$
```

Step 9: Remove Service

```
docker service rm webapp
```

Step 9: Leave Swarm (Optional)

```
docker swarm leave --force
```

Expected Output

- ✓ Swarm initialized
- ✓ Service created
- ✓ Application accessible
- ✓ Scaling works